

Threads em Java: Pesquisa e Aplicação

Programação Orientada a Objetos II — Trabalho II (Etapa 1)

Ciência da Computação — 4º Período — UNIFUCAMP

Aluno: Israel

1. O que é uma Thread e qual a diferença entre processo e thread

Uma thread é uma linha de execução dentro de um programa. Um programa em execução é um processo, e esse processo pode ter uma ou várias threads rodando ao mesmo tempo (ou parecendo rodar, quando há mais threads do que núcleos no processador). Mesmo um programa simples em Java já tem pelo menos uma thread, a *main*, que é a que executa o método `main()`.

A principal diferença é o que cada um compartilha. Cada processo tem o seu próprio espaço de memória, isolado dos outros, e por isso a comunicação entre processos é mais trabalhosa e a criação de um processo é mais pesada para o sistema operacional. Já as threads de um mesmo processo compartilham a memória (o *heap*, onde ficam os objetos), mas cada uma tem sua própria pilha de chamadas. Isso torna criar e trocar de thread mais barato, e deixa a troca de dados entre elas muito simples. O preço dessa facilidade é o risco de duas threads mexerem no mesmo dado ao mesmo tempo, assunto dos tópicos 5 e 6.

	Processo	Thread
Memória	Própria e isolada	Compartilhada com as outras threads do processo
Custo de criação	Maior	Menor
Comunicação	Mais complexa (IPC)	Direta, via objetos compartilhados
Risco principal	Pouco compartilhamento	Condições de corrida

2. Formas de criar uma Thread em Java

Existem duas formas. A primeira é **estender a classe Thread** e sobrescrever o método `run()`. A segunda é **implementar a interface Runnable**, também escrevendo o `run()`, e depois entregar o objeto ao construtor de uma Thread.

```
// 1) estendendo Thread
class MeuCarro extends Thread {
    public void run() { System.out.println("correndo"); }
}
new MeuCarro().start();

// 2) implementando Runnable
class MeuCarro implements Runnable {
    public void run() { System.out.println("correndo"); }
}
```

```
new Thread(new MeuCarro()).start();
```

Estender Thread é mais curto e direto, mas como Java não tem herança múltipla de classes, a classe fica impedida de herdar de qualquer outra. Além disso, mistura duas ideias: o que a tarefa faz e o mecanismo que a executa. Implementar Runnable deixa a classe livre para herdar de outra, separa a tarefa de quem a executa e permite reaproveitar o mesmo objeto em um pool de threads, por exemplo. A desvantagem é uma linha a mais para criar a Thread.

No simulador, escolhi **Runnable** para a classe **Carro**. A classe só precisa descrever o comportamento do carro, e é a classe **Corrida** que decide quando criar e iniciar as threads. Assim a modelagem orientada a objetos fica mais limpa, e o Carro continua podendo herdar de uma classe como Veículo no futuro.

3. Ciclo de vida de uma Thread

Segundo a documentação da enumeração `Thread.State`, uma thread passa por seis estados:

NEW: a thread foi criada com `new Thread(...)`, mas o `start()` ainda não foi chamado.

RUNNABLE: depois do `start()`, a thread está executando ou pronta para executar, esperando o escalonador dar a ela tempo de processador.

BLOCKED: a thread está esperando para entrar em um bloco ou método `synchronized` cujo lock está com outra thread.

WAITING: espera sem prazo definido, por exemplo ao chamar `join()` sem tempo ou `wait()`, até outra thread agir.

TIMED_WAITING: espera com prazo, como em `sleep(ms)` ou `join(ms)`. É onde os carros ficam a maior parte do tempo.

TERMINATED: o `run()` terminou, normalmente ou por exceção. Uma thread terminada não pode ser reiniciada.

Resumindo o caminho típico de um carro: NEW, depois RUNNABLE ao receber o `start()`, alternando com TIMED_WAITING a cada `sleep()`, e por fim TERMINATED ao cruzar a linha de chegada. A thread *main*, enquanto espera os carros no `join()`, fica em WAITING.

4. Métodos fundamentais

`start()` pede à JVM que crie uma nova linha de execução e chame o `run()` nela. `run()` é o corpo da tarefa, o código que a thread executa. `sleep(ms)` é um método estático que pausa a thread atual pelo tempo informado, sem consumir processador, e pode

lançar `InterruptedException`, que precisa ser tratada. `join()` faz a thread que o chama esperar até a outra terminar.

A diferença entre `start()` e `run()` é importante. Chamar `start()` cria de fato uma nova thread, e o `run()` executa nela, em paralelo com o resto do programa. Chamar `run()` diretamente é só uma chamada de método comum: nada é criado, e o código roda na thread que fez a chamada. No simulador, se eu chamasse `run()` em vez de `start()`, o Carro 1 faria a corrida inteira sozinho, depois o Carro 2, e assim por diante, sem nenhuma saída intercalada no console. Além disso, chamar `start()` duas vezes na mesma thread lança `IllegalThreadStateException`.

```
for (Thread t : threads) t.start();    // largada paralela
for (Thread t : threads) t.join();     // main espera todos chegarem
```

5. Condições de corrida (race conditions)

Uma condição de corrida acontece quando duas ou mais threads acessam o mesmo dado compartilhado ao mesmo tempo, pelo menos uma delas alterando-o, e o resultado final depende da ordem em que o escalonador as intercala. O resultado muda de uma execução para outra, o que torna o erro difícil de reproduzir.

O motivo é que operações aparentemente simples não são atômicas. O comando `contador++` na verdade são três passos: ler o valor, somar um e escrever de volta. Se duas threads leem o mesmo valor antes de qualquer uma escrever, uma das atualizações se perde. Exemplo:

```
class Contador {
    int valor = 0;
    void incrementar() { valor++; }    // não é atômico
}

Contador c = new Contador();
Runnable tarefa = () -> { for (int i = 0; i < 100000; i++) c.incrementar(); };
Thread t1 = new Thread(tarefa), t2 = new Thread(tarefa);
t1.start(); t2.start();
t1.join(); t2.join();
System.out.println(c.valor);    // esperado 200000, mas costuma sair menos
```

No simulador o mesmo problema apareceria no pódio. Se dois carros chegassem quase juntos e cada um fizesse "leia o tamanho da lista, use como posição, adicione na lista" sem proteção, os dois poderiam receber o mesmo 1º lugar, ou um deles sobrescrever o registro do outro.

6. Mecanismos de sincronização

A palavra-chave `synchronized` garante exclusão mútua: só uma thread por vez executa o trecho protegido para um mesmo objeto, pois cada objeto Java tem um lock (monitor).

Uma thread que tenta entrar enquanto o lock está ocupado fica em BLOCKED até ele ser liberado. Pode ser usada de duas formas:

```
// método sincronizado: o lock é o próprio objeto (this)
public synchronized void incrementar() { valor++; }

// bloco sincronizado: protege só o trecho crítico
public void registrar(String nome) {
    synchronized (this) {
        lista.add(nome);
    }
}
```

O bloco é preferível quando só uma parte do método precisa de proteção, porque mantém o lock por menos tempo e deixa as threads mais livres. No projeto, a classe `Podio` usa um bloco `synchronized` no método `registrarChegada()`: adicionar o carro à lista e descobrir sua posição acontecem juntos, sem que outro carro se intrometa, então cada carro recebe uma colocação diferente.

Também existem alternativas do pacote `java.util.concurrent`. `AtomicInteger` oferece operações como `incrementAndGet()` que são atômicas sem precisar de lock explícito, e `ReentrantLock` funciona como um `synchronized` manual, com `lock()` e `unlock()`, oferecendo mais controle (como tentar obter o lock sem ficar bloqueado), mas exigindo liberar o lock em um `finally`. Neste trabalho usei apenas `synchronized`, por ser suficiente e mais simples.

7. Conclusão

Ao estudar o tema, a parte que mais exigiu atenção foi entender que o `start()` não executa o código diretamente: ele cria a thread, e o momento em que cada uma roda é decidido pelo escalonador, não por mim. Por isso a saída do simulador muda a cada execução, o que a princípio parece um bug, mas é justamente a prova de que há paralelismo. Também foi difícil perceber que uma condição de corrida pode simplesmente não aparecer em vários testes e depois aparecer, já que depende de coincidências de tempo.

Apreendi a diferença prática entre `start()` e `run()`, a escolher `Runnable` em vez de estender `Thread` por causa da separação de responsabilidades, a usar `join()` para esperar o fim da corrida antes de mostrar o pódio e a proteger o trecho crítico com `synchronized`. O trabalho mostrou que programação concorrente traz ganho de desempenho e de organização, mas exige cuidado com o que é compartilhado entre as threads.

Referências

ORACLE. The Java Tutorials: Lesson — Concurrency. Disponível em:
docs.oracle.com/javase/tutorial/essential/concurrency.

ORACLE. Java SE API Documentation: classes Thread, Thread.State e Runnable (java.lang).

ORACLE. Java SE API Documentation: pacote java.util.concurrent (AtomicInteger, ReentrantLock).

Material e enunciado da disciplina de Programação Orientada a Objetos II.